AWS EKS 기반 PetClinic 3티어 서비스 구축·운영 보고서

작성일 2026-10-28 · 오민희

목차

장	주요 절
0. 요약	아키텍처 그림, 핵심 결과
1. 목표와 설계 기준	1.1 목표와 범위, 1.2 설계 기준
2. 아키텍처와 설계 결정	2.1 외부 진입, 2.2 네트워크, 2.3 클러스터와 접속, 2.4 데이터, 2.5 비밀 관리
3. 구현	3.1 스크립트와 진행, 3.2 EKS 클러스터, 3.3 컨테이너 빌드와 ECR, 3.4 매니페스트, 3.5 배포
4. 검증	4.1 V1 외부 접속, 4.2 V2 무중단 롤링, 4.3 V3 부하, 4.4 그 밖의 검증
5. 심화 실험	5.1 직접 연결 vs RDS Proxy, 5.2 수직 묶음 카나리, 5.3 PITR 복구 리허설
6. 운영	6.1 운영 절차, 6.2 용량 산정, 6.3 보안 점검, 6.4 비용과 철거
7. 회고와 개선 과제	배운 점, 한계와 개선
부록	A 리소스 명명, B 버전, C 스크립트 목록, D web nginx 설정, E 트러블슈팅

0. 요약

spring-framework-petclinic(WAR)을 AWS EKS 위 3티어(web, was, DB)로 올리고, 실제 운영 서비스로 버티는지 측정으로 확인했다. 1인 프로젝트로 설계부터 철거까지 진행했고, 스크립트 초안과 문서 검토에 AI(Claude)를 활용했다.

기술 스택: AWS(EKS 1.36, ECR, RDS MySQL 8.4 Multi-AZ, ALB, Secrets Manager, SSM), Kubernetes(Gateway API, HPA, PDB, NetworkPolicy, Pod Identity), AWS Load Balancer Controller, External Secrets, Helm, Docker(nginx, Tomcat 11), Trivy, k6, eksctl, Bash

저장소: github.com/minhee-oh/petclinic-eks (스크립트, 매니페스트, Dockerfile 전체)

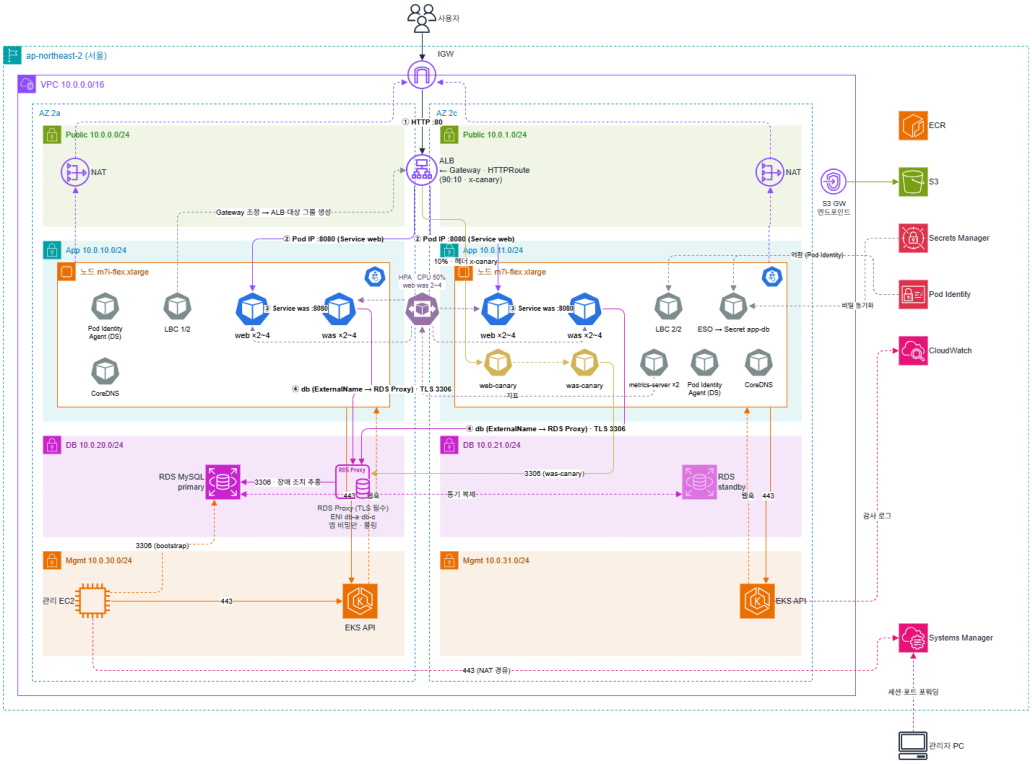


그림 0-1. 전체 아키텍처

- **핵심 검증 V1~V9 모두 통과.** V1, V2, V3는 문제를 찾아 고친 뒤 다시 측정해 통과했다(아래 표, 4장)
- **부하**: 474,912건 실패 0, p95 92ms (초당 약 1,580 요청), HPA 2→4→2
- **무중단**: 롤링 배포 3회, 노드 drain과 재배치 중 요청 983건에서 200 아닌 응답 0
- **DB 장애 조치**: 앱 영향 약 10초, Pod 재시작 0. RDS Proxy를 넣어 빨라졌다는 근거는 없었다(5.1)
- **비용**: 실제 약 $32 (Cost Explorer, 10/22~10/27). 24시간 운영 기준으로는 월 약 $920이다(6.4). 10/26에 모두 철거했고 최종 스냅샷만 남겼다

측정으로 찾아 고친 문제

검증	발견한 문제	해결
V1 외부 접속	was 기동 실패: DB 주소가 빌드 때 `localhost`로 고정됨	자리표시 키 변경, 1.0.1 재빌드 (4.1)
V2 무중단 롤링	롤링 후 web 2개가 한 AZ에 모임	topologySpread에 `matchLabelKeys`, AZ 분산 확인 (4.2)
V3 부하	Target 5XX 3.5%	nginx upstream keepalive, 1.0.3에서 0 (4.3)

1. 목표와 설계 기준

1.1 목표와 범위

원본 petclinic은 정적 파일과 앱을 WAR 하나로 내보내는 단일 애플리케이션이다. 이것을 web(nginx)과 was(Tomcat) 컨테이너로 나눠 EKS에 올리고, DB는 컨테이너 밖 RDS에 둔다. 목표는 서비스를 띄우는 데서 끝내지 않고, 아래 항목을 측정으로 확인하는 것이다.

목표	확인 방법	위치
외부에서 전 구간 동작	ALB → web → was → RDS 200, 브라우저에서 등록과 조회	4.1
배포와 노드 정비 중 무중단	롤링 3회와 drain 중 curl 루프	4.2
부하에 따른 자동 확장	k6 부하 + HPA	4.3
DB 장애에도 복구	RDS 장애 조치, PITR 복원	5.1, 5.3
최소 권한, 비밀 관리, 격리	NetworkPolicy 차단 시험(V5), 보안 점검표	4.4, 6.3
처음부터 다시 만들 수 있음	철거 후 같은 날 스크립트로 재구축	3.1

web과 was는 이미지, Deployment, Service, HPA를 따로 가진 독립 배포 단위다. 카나리는 새 web이 새 was로만 가도록 둘을 한 묶음으로 올린다(5.2).

- 범위 밖: 앱을 서비스별로 쪼갠 spring-petclinic-microservices로 분리, HTTPS(도메인 없음), 별도 모니터링 스택
- 기간: 2026-10-22 ~ 10-26

1.2 설계 기준

- **실제 운영 서비스로 가정한다.** 편의를 위한 선택은 운영 기준으로 설명될 때만 채택했다.
- **24시간 가동**을 전제로 설계한다. 밤에 끈 것은 테스트 기간의 비용 조치일 뿐이다(6.4).
- **AZ 2개, N+1.** AZ 하나 또는 노드 한 대가 빠져도 서비스가 유지돼야 한다.

2. 아키텍처와 설계 결정

결정마다 검토했지만 쓰지 않은 대안과 그 이유를 함께 적었다.

2.1 외부 진입 (Ingress 대신 Gateway API)

AWS Load Balancer Controller(LBC)가 만드는 ALB가 Pod IP로 직접 보낸다. 경로, 가중치, 헤더 라우팅은 Gateway API의 HTTPRoute로 처리한다. Ingress는 가중치와 헤더 라우팅을 컨트롤러별 어노테이션으로만 표현하지만, HTTPRoute에서는 표준 스펙 필드다. 채택 전에 샘플 Gateway로 가중치와 헤더 라우팅을 확인했다.

- **NLB + ingress-nginx를 쓰지 않은 이유**: ALB가 L7을 직접 처리하므로 클러스터 안 프록시 Pod를 따로 운영할 이유가 없다. 또 ingress-nginx는 2026년 3월로 유지보수가 끝났다(Kubernetes 공식 블로그).

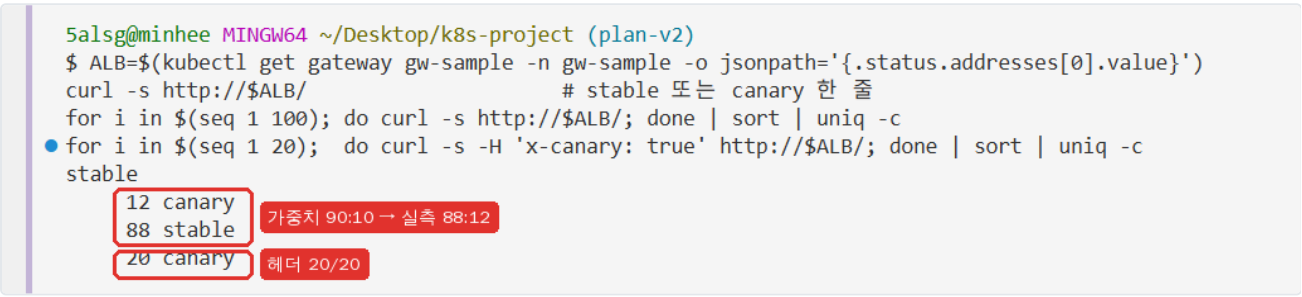


그림 2-1. 샘플 Gateway 확인. 가중치 90:10 설정에 실측 88:12, x-canary 헤더 20/20

2.2 네트워크

구역	AZ a / c 대역	들어가는 것	기본 경로
public	10.0.0.0/24, 10.0.1.0/24	ALB, NAT(AZ마다 1개)	IGW
private-app	10.0.10.0/24, 10.0.11.0/24	EKS 노드 2대	같은 AZ의 NAT
private-db	10.0.20.0/24, 10.0.21.0/24	RDS(Multi-AZ), RDS Proxy	없음
private-mgmt	10.0.30.0/24, 10.0.31.0/24	SSM 관리 EC2, EKS 컨트롤 플레인 ENI	같은 AZ의 NAT

- **NAT는 AZ마다 1개** 둔다. 한 AZ가 죽어도 다른 AZ의 나가는 길은 산다.
- ECR 이미지 레이어는 S3에서 내려받는다. 그래서 S3 게이트웨이 엔드포인트를 app과 mgmt 라우트 테이블에 붙여 NAT를 거치지 않게 했다(NAT 데이터 처리 요금 절감).
- **DB 서브넷에는 인터넷 경로가 없다.** 격리는 SG와 NetworkPolicy가 맡는다.
- RDS 3306은 `pc-sg-node`, `pc-sg-mgmt`, `pc-sg-proxy`에서만, EKS 443은 `pc-sg-mgmt`에서만 연다. 관리 EC2는 인바운드 규칙이 0개다.
- **리전 NAT 게이트웨이를 쓰지 않은 이유**: AZ 2개 고정이라 자동 AZ 확장이 필요 없고, ALB 때문에 퍼블릭 서브넷이 어차피 필요하다. AZ 장애 시 동작이 문서로 확인되는 쪽은 존별 NAT다.

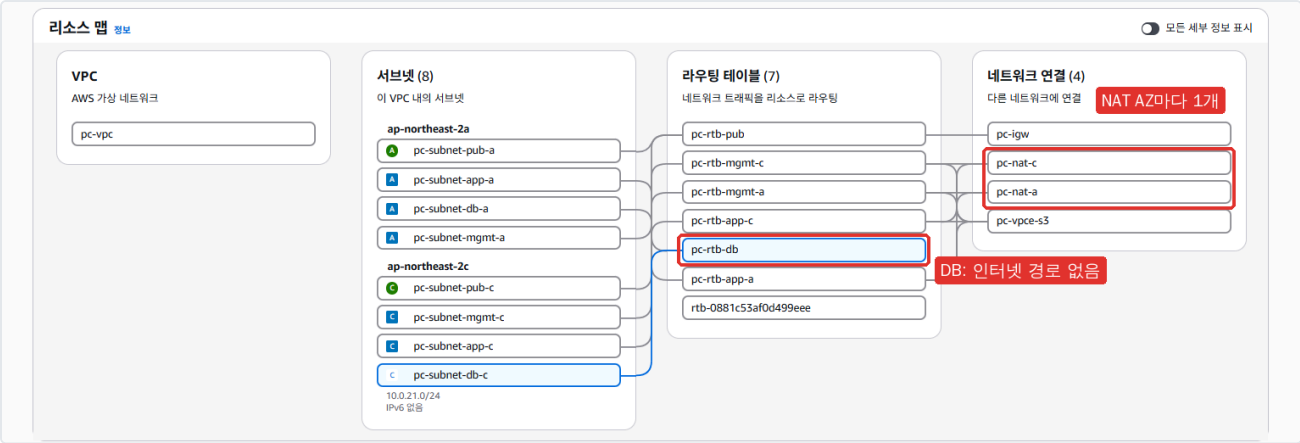


그림 2-2. VPC 리소스 맵

2.3 클러스터와 접속

EKS는 쿠버네티스의 컨트롤 플레인을 AWS가 운영하는 관리형 서비스다. 역할을 아래처럼 나눴다.

영역	AWS가 맡는 것	내가 정한 것
컨트롤 플레인	API 서버와 etcd를 여러 AZ에 운영하고 패치한다	쿠버네티스 버전(1.36)과 올릴 시점, API 엔드포인트 공개 범위(프라이빗), 인증 방식(액세스 엔트리), ENI를 둘 서브넷
노드	관리형 노드 그룹: EKS 최적화 AMI를 제공하고, 업데이트할 때 drain 후 롤링 교체를 대신 수행한다	업데이트 시점, 인스턴스 유형과 수, 서브넷, IMDS 제한, 볼륨 암호화
네트워크	VPC CNI가 Pod에 VPC IP를 준다(ALB가 Pod IP로 바로 보낼 수 있는 이유)	VPC와 서브넷 설계, NetworkPolicy 켜기
애드온	EKS 애드온(vpc-cni, metrics-server 등)을 클러스터 버전과의 호환성을 검증해 제공한다	어느 애드온을 켤지, 버전 선택과 설정 값(NetworkPolicy 켜기 등). LBC와 External Secrets Operator(ESO)는 Helm으로 직접 설치
IAM	Pod Identity로 Pod에 임시 자격 증명 발급	어느 ServiceAccount에 어느 역할을 줄지

- **EKS API는 프라이빗 엔드포인트만** 연다. 로컬 kubectl은 SSM 포트 포워딩으로만 붙는다.
- 로컬(VPC 밖)에서 eksctl로 만들려면 생성 중에 API에 닿아야 해서, 생성 중에만 내 공인 IP /32로 퍼블릭 엔드포인트를 열고 직후 닫는다.
  - **eksctl 완전 프라이빗 모드를 쓰지 않은 이유**: 관리 EC2에 클러스터, CloudFormation, IAM 생성 권한이 필요해 최소 권한에 어긋난다. 생성 중 /32 노출(20~30분)이 위험이 더 작다.
- **노드는 관리형 노드 그룹 m7i-flex.xlarge 2대(AZ마다 1대) 고정.** HPA 최대치를 노드 1대에 올리는 N+1로 잡았다(6.2).
  - **Karpenter를 쓰지 않은 이유**: N+1 구성이라 노드 자동 확장이 필요 없다고 판단했다(시스템 Pod를 뺀 추정, 6.2).
  - **EKS Auto Mode를 쓰지 않은 이유**: 노드와 로드밸런서를 AWS가 관리해 Gateway API와 LBC를 직접 다루는 목표에서 멀어진다.
- **Pod IAM은 Pod Identity**로 LBC와 ESO에만 준다. 노드 IMDS는 hop limit 1이라 Pod가 노드 역할을 물려받지 못한다.

관리 EC2 `pc-mgmt` (t3.medium)는 관리 서브넷에 두었다. 퍼블릭 IP와 키 페어가 없고, IMDSv2를 필수로 하며 루트 볼륨을 암호화했다. 22번 포트를 열지 않고 SSM으로만 들어간다(Bastion 대신 SSM).

```
kubectl ──https://<엔드포인트 이름>:8443──▶ 127.0.0.1:8443 (hosts 파일)
        ──SSM 포트 포워딩──▶ pc-mgmt ──443──▶ EKS 프라이빗 엔드포인트
```

hosts 파일에 엔드포인트 이름을 넣은 것은 API 서버 인증서가 그 이름으로 발급돼 있기 때문이다. IP로 붙으면 TLS 검증이 실패한다.

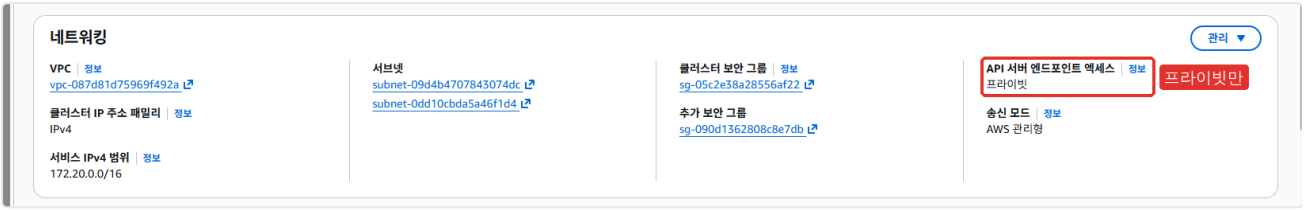


그림 2-3. EKS 네트워킹: API 서버 엔드포인트 액세스 프라이빗

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ kubectl get nodes -o wide
NAME                                              STATUS   ROLES    AGE   VERSION               INTERNAL-IP    EXTERNAL-IP   OS-IMAGE
        KERNEL-VERSION                    CONTAINER-RUNTIME
ip-10-0-10-62.ap-northeast-2.compute.internal     Ready    <none>   59m   v1.36.5-eks-117f323   10.0.10.62     <none>        Amazon Linux 2023.12.20
261016   6.18.54-121.166.amzn2023.x86_64 (amd64)  containerd://2.3.5+unknown
ip-10-0-11-155.ap-northeast-2.compute.internal    Ready    <none>   59m   v1.36.5-eks-117f323   10.0.11.155    <none>        Amazon Linux 2023.12.20
261016   6.18.54-121.166.amzn2023.x86_64 (amd64)  containerd://2.3.5+unknown
```

그림 2-4. 로컬 kubectl get nodes, 2대 Ready (10.0.10.x / 10.0.11.x)

2.4 데이터

- `pc-rds`: MySQL 8.4.11, db.m8g.large, gp3 20GB 암호화, Multi-AZ, 퍼블릭 접근 없음
- 자동 백업 7일, 삭제 방지, master 비밀번호는 RDS가 Secrets Manager에 직접 관리
- 파라미터 그룹 `pc-mysql84`: `require_secure_transport=1`로 TLS 없는 접속을 거부한다
- **버스터블 RDS(db.t4g)를 쓰지 않은 이유**: RDS t4g는 Unlimited 모드라 크레딧이 바닥나도 성능은 유지되지만, 기준 성능을 넘긴 시간만큼 추가 요금이 붙어 비용이 부하에 따라 흔들린다. 메모리도 작아 `max_connections`가 확장 상한이 된다. db.m8g.large로 올리면 이번 기간 약 $13이 더 드는 것으로 추정했다.

DB 계정은 용도별로 나눴다. 앱 계정은 DML만 가진다.

```
CREATE USER 'pc_app'@'%' IDENTIFIED BY '…' REQUIRE SSL;
GRANT SELECT, INSERT, UPDATE, DELETE ON `petclinic`.* TO 'pc_app'@'%';
CREATE USER 'pc_migration'@'%' IDENTIFIED BY '…' REQUIRE SSL;      -- 초기화 Job 전용
GRANT CREATE, ALTER, DROP, INDEX, REFERENCES, SELECT, INSERT, UPDATE, DELETE ON `petclinic`.* TO 'pc_m
   igration'@'%';
```

앱은 DB를 `db`라는 이름(ExternalName Service)으로만 안다. RDS Proxy나 PITR 복원본으로 바꿀 때 이 한 줄만 바꾸면 되고, 이미지를 다시 빌드하지 않는다.

- **RDS Proxy를 상시 쓰지 않은 이유**: 측정에서 장애 조치가 빨라졌다는 근거는 없었다(5.1). Pod가 늘어 커넥션이 상한이 될 때 도입한다.

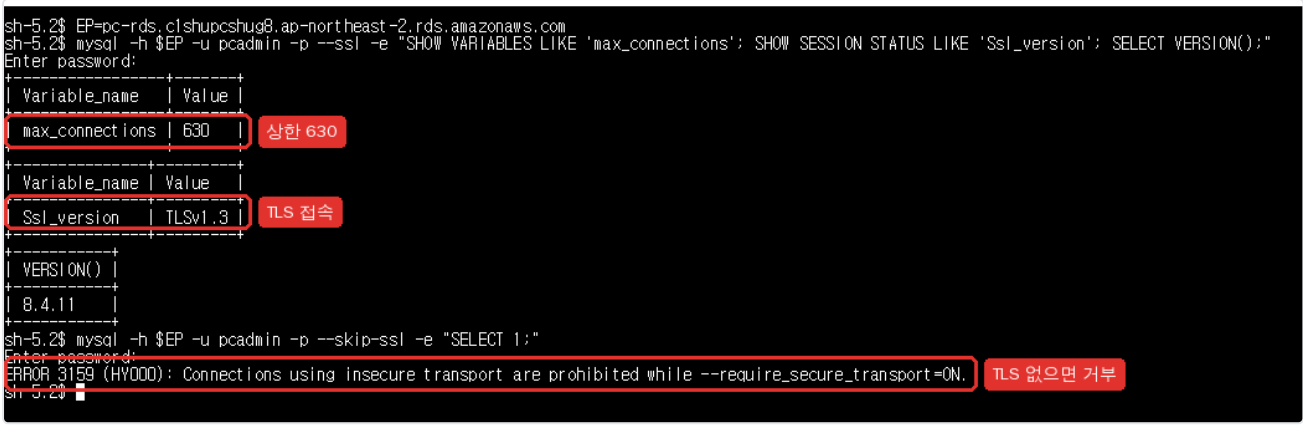


그림 2-5. max_connections 630, --ssl 접속 TLSv1.3 / --skip-ssl 접속 ERROR 3159

2.5 비밀 관리 (Secrets Manager + External Secrets)

비밀 원본은 Secrets Manager에 두고 ESO로 동기화한다. 비밀번호가 명령 기록, 저장소 파일, 화면에 남지 않게 했다(실행 중인 was 컨테이너 안 설정 파일은 예외, 3.3). 값은 AWS가 무작위로 만들고, RDS master 비밀은 클러스터에 들어오지 않는다.

```
pw=$(aws secretsmanager get-random-password --password-length 32 --exclude-punctuation \
      --require-each-included-type --query RandomPassword --output text)
aws secretsmanager create-secret --name pc/petclinic/app-db \
  --secret-string "{\"username\":\"pc_app\",\"password\":\"$pw\"}"; unset pw
```

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata: {name: app-db, namespace: petclinic}
spec:
  refreshInterval: 1h
  secretStoreRef: {name: aws-sm, kind: SecretStore}      # ESO 자신의 Pod Identity 로 읽는다
  target: {name: app-db, creationPolicy: Owner}
  dataFrom: [{extract: {key: pc/petclinic/app-db}}]
```

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/10-secrets.yaml
kubectl get secretstore,externalsecret -n petclinic
kubectl get secret -n petclinic
namespace/petclinic created
secretstore.external-secrets.io/aws-sm created
externalsecret.external-secrets.io/app-db created
externalsecret.external-secrets.io/migration-db created
NAME                                  AGE   STATUS   CAPABILITIES   READY
secretstore.external-secrets.io/aws-sm   2s    Valid    ReadWrite      True

NAME                                          STORETYPE     STORE    REFRESH INTERVAL   STATUS         READY   LAST SYNC
externalsecret.external-secrets.io/app-db         SecretStore   aws-sm   1h                 SecretSynced   True    2s
externalsecret.external-secrets.io/migration-db   SecretStore   aws-sm   1h                 SecretSynced   True    2s
NAME           TYPE     DATA   AGE
app-db         Opaque   2      3s
migration-db   Opaque   2      2s
```
동기화 완료

그림 2-6. ExternalSecret 2개 SecretSynced

3. 구현

3.1 스크립트와 진행

시작 전에 `01-preflight.sh`가 자격 증명, 할당량, AZ, 인스턴스 유형, RDS 엔진과 클래스 조합을 조회만으로 확인한다. 모든 자원은 로컬 Git Bash에서 스크립트(`scripts/NN-*.sh`, 부록 C)로 만들었다. 관리 EC2에서 할 일(DB 계정 생성, 이미지 빌드)도 로컬 스크립트가 SSM Run Command로 보낸다. 모든 생성 스크립트는 이름 태그로 자원을 먼저 찾고 없을 때만 만든다. 그래서 중간에 끊겨도 다시 실행하면 이어서 진행한다.

작업 순서는 아래와 같다. 괄호는 스크립트 번호다.

```
사전 점검(01) → 네트워크(10) → 관리 EC2(20) → RDS(31) → EKS(30) → SSM 터널(21) → 애드온(40)
→ 비밀·ESO(35) → DB 계정(32) → ECR(34) → 빌드·스캔·push(60) → 배포(70)
→ 검증 V1~V9(90) → 구성 덤프(80) → 철거(destroy)
→ 스크립트만으로 재구축 → 카나리(75)·PITR(77)·RDS Proxy(76) → 최종 철거
```

철거한 뒤 같은 날 스크립트만으로 전체를 다시 만들었다. 17:33(KST)에 시작해 19:13에 ALB 200으로 돌아왔다(약 1시간 40분, PC가 꺼져 멈춘 약 30분 포함). 이 과정에서 두 가지가 빠졌다. Pod Identity 연결은 생성 중 PC가 꺼져 만들어지지 않아 수동으로 만들었고(T10), metrics-server는 eksctl 기본값에 기대고 있어 템플릿에 명시했다(T11). 최종 철거는 재시도 없이 한 번에 끝났다.

3.2 EKS 클러스터

`30-cluster.sh`는 생성 → 컨트롤 플레인 ENI를 mgmt 서브넷으로 → 프라이빗 전환 → 443 허용 순서로 진행하고, 마지막에 노드마다 SG와 IMDS 설정을 검사해 다르면 중단한다.

```yaml
# infra/cluster.yaml.tpl 발췌
accessConfig:
  authenticationMode: API                # aws-auth ConfigMap 미사용
vpc:
  controlPlaneSubnetIDs: [<mgmt-a>, <mgmt-c>]
  clusterEndpoints: {privateAccess: true, publicAccess: true}   # 생성 중에만
addons:
  - name: vpc-cni
    configurationValues: '{"enableNetworkPolicy": "true"}'      # 없으면 NetworkPolicy 가 아무것도 막지 않음
  - {name: metrics-server}               # 명시하지 않으면 재구축 때 빠졌다 (T11)
iam:
  podIdentityAssociations:
    - {namespace: kube-system, serviceAccountName: aws-load-balancer-controller, roleName: pc-role-lb
   c}
    - {namespace: external-secrets, serviceAccountName: external-secrets, roleName: pc-role-eso}
managedNodeGroups:
  - name: pc-ng-app
    instanceType: m7i-flex.xlarge
    desiredCapacity: 2
    volumeEncrypted: true
    disableIMDSv1: true
    disablePodIMDS: true                 # hop limit 1
    privateNetworking: true
```

3.3 컨테이너 빌드와 ECR

원본 WAR를 역할별 컨테이너 두 개로 나눴다.

이미지	베이스	하는 일
web	`nginx-unprivileged:1.30.6-alpine`	WAR 빌드 결과의 정적 파일(`/resources`)을 직접 내보내고, 나머지 요청만 was로 넘긴다. 비루트, 26MB
was	`tomcat:11.0.27-jre21-temurin-noble`	WAR를 Tomcat에서 실행한다. DB 주소와 계정은 실행 시 주입한다. 비루트, 235MB

원본에서 바꾼 것 3가지 (빌드 스크립트가 clone 직후 수정하고, 수정이 안 됐으면 중단한다)

위치	바꾼 내용	이유
`pom.xml`	MySQL 드라이버를 `com.mysql:mysql-connector-j`로	원본의 옛 좌표에는 요구 버전이 없어 빌드가 실패한다 (T1)
`datasource-config.xml`	접속 정보 자리표시를 `${jdbc.*}` → `${pc.jdbc.*}`	원본 키는 Maven 필터링이 빌드 때 `localhost`로 박아 실행 시 바꿀 수 없다 (T2, T3)
`datasource-config.xml`	풀 상한 8, 빌려줄 때 `SELECT 1` 검사	DB 연결 수를 계산 가능하게(6.2), 장애 조치 뒤 끊긴 연결을 버리게(5.1)

was 이미지

```
FROM tomcat:11.0.27-jre21-temurin-noble
RUN apt-get update && apt-get -y upgrade && rm -rf /var/lib/apt/lists/* /usr/local/tomcat/webapps.dist
COPY ROOT.war /usr/local/tomcat/webapps/ROOT.war
COPY --chmod=755 pc-entrypoint.sh /usr/local/bin/pc-entrypoint.sh
RUN useradd -r -u 10001 -g 0 -d /usr/local/tomcat tomcat \
 && chown -R 10001:0 /usr/local/tomcat && chmod -R g=u /usr/local/tomcat
USER 10001
ENV JAVA_OPTS="-XX:MaxRAMPercentage=70 -Dsun.net.inetaddr.ttl=30"
EXPOSE 8080
CMD ["/usr/local/bin/pc-entrypoint.sh"]
```

- WAR는 관리 EC2에서 Maven 컨테이너로 만든다. 이미지에는 Maven과 JDK가 들어가지 않는다.
- `ROOT.war`로 넣어 주소가 `/`가 되고, 쓰지 않는 기본 앱 원본(`webapps.dist`)은 지웠다.
- 힙은 컨테이너 메모리의 70%로 잡았다. DNS 캐시를 30초로 줄인 것은 RDS 장애 조치 때 엔드포인트 IP가 바뀌기 때문이다.

엔트리포인트는 환경 변수(`DB_HOST`는 `db`, `DB_USER`와 `DB_PASS`는 Secret)를 `conf/catalina.properties` 끝에 덧붙이고, `DB_PASS`를 지운 뒤 Tomcat을 띄운다. Tomcat은 이 파일의 항목을 시스템 속성으로 읽고, 앱 설정의 `${pc.jdbc.*}` 자리를 그 값으로 채운다.

- **이 방식을 고른 이유**: `-D` 옵션으로 넘기면 비밀번호가 Tomcat 시작 로그와 `ps` 출력에 찍힌다.
- **한계**: 비밀번호가 컨테이너 안 이 파일에 평문으로 남는다. 컨테이너의 쓰기 계층이라 Pod가 지워지면 같이 사라지고 이미지에는 들어가지 않지만, `kubectl exec` 권한이 있으면 읽을 수 있다. 운영에서는 이 권한을 제한해야 한다.

```
# pc-entrypoint.sh 발췌
cat >> /usr/local/tomcat/conf/catalina.properties <<PROPS
pc.jdbc.url=jdbc:mysql://${DB_HOST}:3306/petclinic?useUnicode=true&sslMode=REQUIRED
pc.jdbc.username=$(esc "$DB_USER")
pc.jdbc.password=$(esc "$DB_PASS")
jdbc.initLocation=file:/usr/local/tomcat/conf/pc-noop.sql
jdbc.dataLocation=file:/usr/local/tomcat/conf/pc-noop.sql
PROPS
unset DB_PASS
exec catalina.sh run
```

원본 앱은 기동할 때마다 스키마 SQL을 실행한다. 앱 계정은 DML만 가져 CREATE에서 실패하므로, 스키마와 초기 데이터는 마이그레이션 계정을 쓰는 초기화 Job으로 나누고, 앱에는 `initLocation`과 `dataLocation`을 `SELECT 1`만 든 파일로 바꿔 달았다.

web 이미지

```
# BASE_IMAGE 는 V9 시험 때만 옛 이미지로 바꾼다
ARG BASE_IMAGE=nginxinc/nginx-unprivileged:1.30.6-alpine
FROM ${BASE_IMAGE}
ARG APP_VERSION=1.0.0
ENV APP_VERSION=${APP_VERSION}
# 카나리 web 은 Deployment env 로 was-canary 를 준다
ENV WAS_UPSTREAM=was
# 시작 시 환경 변수를 치환해 default.conf 를 만든다
COPY default.conf.template /etc/nginx/templates/default.conf.template
# WAR 빌드 결과의 정적 파일
COPY resources /usr/share/nginx/html/resources
EXPOSE 8080
```

- 비루트(uid 101) 이미지라 8080에서 듣는다.
- 프록시할 was 이름과 앱 버전을 환경 변수로 받는다. 같은 이미지가 stable에서는 `was`, 카나리에서는 `was-canary`로 보낸다(5.2). 응답 헤더 `X-App-Version`으로 어느 버전이 답했는지 보인다.
- was로 가는 연결을 재사용한다(upstream keepalive). 요청마다 새 연결을 열면 부하 중 502가 난다(4.3). nginx 설정은 부록 D에 있다.

빌드 관문

로컬의 `60-build-push.sh`가 SSM으로 관리 EC2에서 아래 순서로 실행하고, 이어서 `70-deploy.sh`가 배포한다(3.5). 주황 테두리 단계는 관문이라, 조건을 못 넘으면 그 자리에서 멈추고 push와 배포를 하지 않는다.



그림 3-1. 빌드와 배포 파이프라인

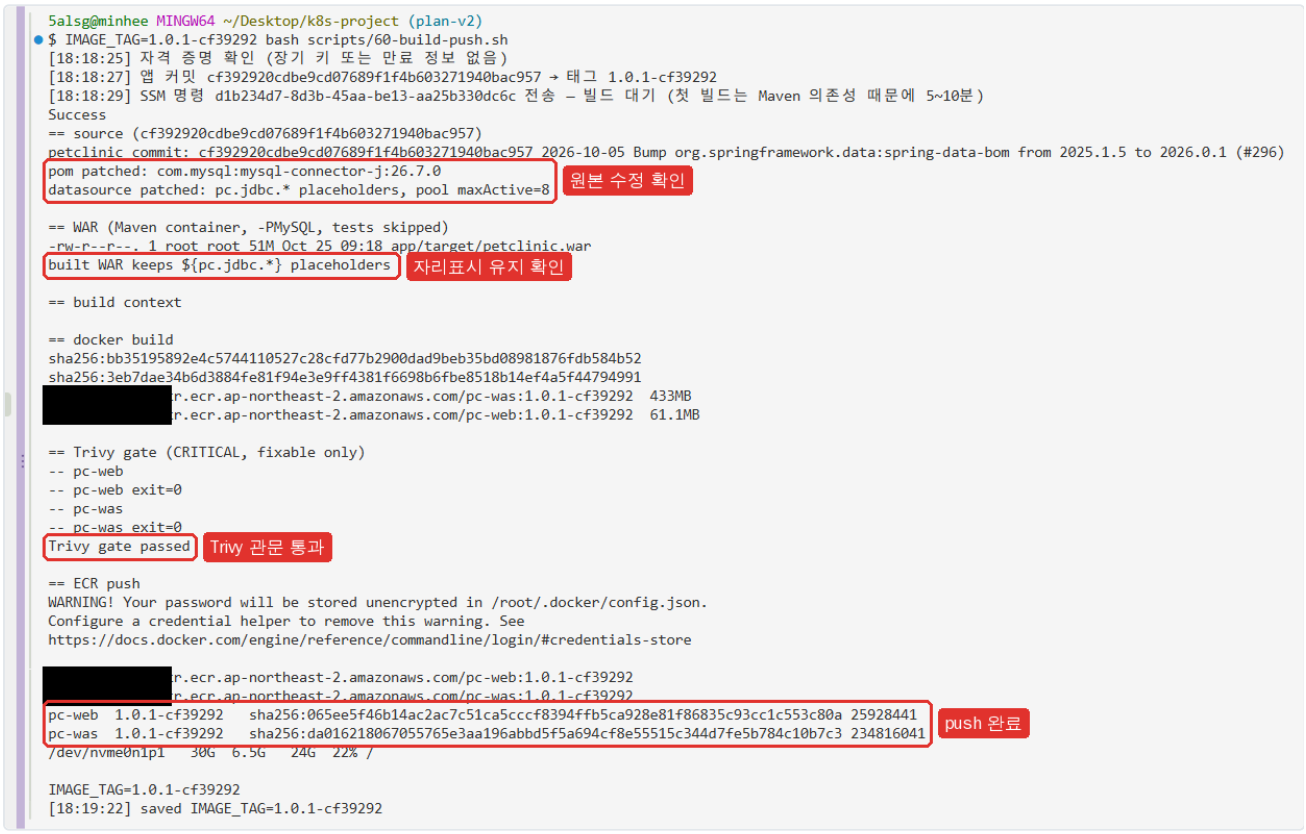


그림 3-2. 빌드 → Trivy 관문 통과 → ECR push (계정 ID 가림)

ECR

`pc-web`과 `pc-was` 두 저장소를 만들었다.

- 태그는 변경할 수 없다(IMMUTABLE). 태그는 `<앱 버전>-<커밋>`
- AES256으로 암호화하고 향상된 연속 스캔을 켰다.
- 태그 없는 이미지는 1일 뒤 지우고, 최근 10개만 남긴다.
- push는 관리 EC2 역할만 할 수 있다.

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/34-ecr.sh
[16:33:25] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[16:33:27] ECR pc-web 생성
[16:33:30] ECR pc-was 생성
[16:33:32] 향상된 스캔(ENHANCED, CONTINUOUS_SCAN, pc-*) 설정
-------------------------------------------------------------------------------------------
|                                   DescribeRepositories                                  |
+--------+-------------+----------+-----------------------------------------------------+
|  pc-web|  IMMUTABLE  |  AES256  | 태그 불변, 암호화 .dkr.ecr.ap-northeast-2.amazonaws.com/pc-web  |
|  pc-was|  IMMUTABLE  |  AES256  |                   .dkr.ecr.ap-northeast-2.amazonaws.com/pc-was  |
+--------+-------------+----------+-----------------------------------------------------+
ENHANCED        CONTINUOUS_SCAN pc-*   향상된 연속 스캔
```

그림 3-3. ECR 저장소 IMMUTABLE, AES256, 향상된 스캔 (계정 ID 가림)

3.4 매니페스트

매니페스트는 필드마다 이유를 정해 직접 썼다. `__IMAGE__`, `__DB_HOST__` 같은 자리표시는 `70-deploy.sh`가 채운다. 그래서 이미지 주소와 DB 주소가 파일에 고정되지 않는다. 아래는 발췌이고, 전체 파일은 저장소 k8s/에 있다.

그림 3-4는 이 매니페스트들이 클러스터 안에서 어떻게 이어지는지 보여 준다. 사용자 요청은 ALB에서 web Pod IP로 바로 가고(Service web은 그 Pod IP 목록을 LBC에 알려 주는 역할), web → Service was → was → Service db → RDS 순서로 흐른다.



그림 3-4. 클러스터 안 리소스 관계도

파일	내용
`00-namespace`	Pod Security Standards(PSS) `enforce: baseline`, `warn`과 `audit`은 `restricted`, readiness gate 주입 라벨
`10-secrets`	SecretStore, ExternalSecret 2개 (2.5)
`15-services`	web, was ClusterIP, `db` ExternalName
`20-was`, `30-web`	Deployment, PDB, HPA
`40-gateway`	Gateway, 대상 그룹 설정, HTTPRoute
`45-canary`	카나리 Deployment와 Service, 가중치 HTTPRoute
`50-netpol`	기본 차단 + 허용 경로
`60-db-init-job`	스키마 초기화 Job (`pc_migration`)

Deployment, PDB, HPA (20-was.yaml, web도 같은 구조)

무중단과 AZ 분산을 정하는 줄만 뽑았다. 나머지 설정은 아래 표에 이유와 함께 적었다.

```
strategy: {rollingUpdate: {maxSurge: 1, maxUnavailable: 0}}
topologySpreadConstraints:
  - {maxSkew: 1, topologyKey: topology.kubernetes.io/zone, matchLabelKeys: [pod-template-hash], ...}
lifecycle: {preStop: {sleep: {seconds: 15}}}
```

설정	이유
`replicas` 생략	있던 필드를 나중에 지우면 `apply`가 1개로 되돌린다
maxSurge 1, maxUnavailable 0	새 Pod가 Ready 된 뒤에 옛 Pod를 내린다 (V2)
`matchLabelKeys: [pod-template-hash]`	없으면 롤링 중 옛 Pod까지 세어 한 AZ로 몰렸다 (4.2)
CPU 상한 없음, 메모리 요청 = 상한	스로틀링 방지, 힙은 메모리의 70%
startupProbe 150초	기동 중 liveness가 Pod를 죽이지 않게
web probe `/healthz`	was 장애가 web 재시작으로 번지지 않게
preStop 15 < 등록 해제 30 ≤ 유예 45초	Pod가 ALB에서 빠진 뒤 종료된다
PDB minAvailable 1	drain 때 한 번에 다 빠지지 않는다 (V7)
HPA 최대 4	노드 1대가 받을 수 있는 양(N+1, 6.2). DB 연결 계산은 이 값을 입력으로 쓴다

DB를 이름 하나로 (15-services.yaml)

```
kind: Service
metadata: {name: db, namespace: petclinic}
spec:
  type: ExternalName
  externalName: __DB_HOST__        # RDS 또는 RDS Proxy 엔드포인트
```

외부 진입 (40-gateway.yaml)

Gateway는 HTTP 80 리스너 하나고, HTTPRoute는 `/` 를 Service `web` 으로 보낸다. 대상 그룹 설정에서 바꾼 것은 세 가지다.

설정	이유
`targetType: ip`	ALB가 Pod IP로 바로 보낸다. readiness gate를 쓰는 조건
등록 해제 지연 30초	기본 300초면 롤링이 느리다
헬스 체크 `/healthz`	was 상태와 상관없이 web 자체만 본다

수직 묶음 카나리 (45-canary.yaml)

```
kind: HTTPRoute
spec:
  rules:
    - matches: [{headers: [{type: Exact, name: x-canary, value: "true"}]}]   # 헤더 → 카나리 100%
      backendRefs: [{name: web-canary, port: 8080}]
    - matches: [{path: {type: PathPrefix, value: /}}]
      backendRefs:
        - {name: web, port: 8080, weight: __STABLE_W__}
        - {name: web-canary, port: 8080, weight: __CANARY_W__}
# web-canary 의 env WAS_UPSTREAM=was-canary → 새 버전 요청은 새 was 로만 간다
# 카나리 Pod 라벨 name=petclinic-canary → stable Service·HPA·PDB 에 잡히지 않는다
```

NetworkPolicy (50-netpol.yaml)

```
# default-deny: 모든 Pod 의 Ingress·Egress 차단 + allow-dns: kube-dns 53
kind: NetworkPolicy
metadata: {name: was}
spec:
  podSelector: {matchLabels: {tier: was}}
  ingress:
    - from: [{podSelector: {matchLabels: {tier: web}}}]
      ports: [{protocol: TCP, port: 8080}]
  egress:
    - to: [{ipBlock: {cidr: 10.0.20.0/24}}, {ipBlock: {cidr: 10.0.21.0/24}}]   # DB 서브넷
      ports: [{protocol: TCP, port: 3306}]
# web: ALB 서브넷 → 8080, → was 8080 / db-init: → DB 서브넷 3306
```

DB로 가는 규칙은 서브넷 대역으로 적었다. ExternalName은 Pod 셀렉터로 고를 수 없고, 대역으로 적으면 장애 조치로 IP가 바뀌거나 Proxy를 끼워도 정책을 고칠 필요가 없다.

초기화 Job (60-db-init-job.yaml)

- 이름에 SQL과 Job 정의의 해시를 붙였다. 내용이 같으면 다시 돌지 않고, 바뀌면 새 Job이 생긴다.
- 비밀번호는 명령줄에 두지 않고 메모리 위 `/tmp` 의 600 권한 파일로만 넘긴다.
- 루트 파일 시스템은 읽기 전용이고, 이미지는 다이제스트로 고정했다.

3.5 배포

`70-deploy.sh` 는 아래 순서를 지킨다(그림 3-1 아래 줄). 앞 단계가 실패하면 앱을 배포하지 않는다.

1. 네임스페이스와 ExternalSecret 적용, Secret 동기화 대기
2. NetworkPolicy 적용. 초기화 Job보다 먼저 둬서 Job도 정책 아래에서 DB에 닿는지 확인된다
3. 초기화 Job 실행, 완료 대기(실패하면 중단)
4. Service → Gateway, HTTPRoute
5. TargetGroupBinding(TGB) 생성 대기. readiness gate는 Pod가 생길 때 TGB가 있어야 붙는다
6. was → web 순서로 Deployment 적용, 각각 rollout 완료 대기

결과: 초기화 Job(테이블 7, owners 10, pets 13), PSS 경고 0건, readiness gate 1/1

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ kubectl -n petclinic get pod -l tier=web -o wide
kubectl -n petclinic describe pod web-85894547df-94ngf | sed -n '/^Readiness Gates/,/^Volumes/p;/Liveness/p;/Readiness:/p;/Startup/p;/Limits/,/mem
ory/p'
NAME                   READY   STATUS    RESTARTS   AGE     IP            NODE                                               NOMINATED NODE   READIN
ESS GATES
web-85894547df-94ngf   1/1     Running   0          9m43s   10.0.11.132   ip-10-0-11-155.ap-northeast-2.compute.internal   <none>           1/1
web-85894547df-qmw4b   1/1     Running   0          9m27s   10.0.10.236   ip-10-0-10-62.ap-northeast-2.compute.internal    <none>           1/1
    Limits:                                                                                                                                  AZ 2개에 1개씩
      memory:  128Mi
    Liveness:     http-get http://:http/healthz delay=0s timeout=5s period=10s successThreshold=1 failureThreshold=3
    Readiness:    http-get http://:http/healthz delay=0s timeout=2s period=5s successThreshold=1 failureThreshold=2
    Startup:      http-get http://:http/healthz delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=30
Readiness Gates:
  Type                                                Status
  target-health.elbv2.k8s.aws/k8s-petclini-web-12d227a17b   True   readiness gate True
Conditions:
  Type                                                Status
  target-health.elbv2.k8s.aws/k8s-petclini-web-12d227a17b   True
  PodReadyToStartContainers                           True
  Initialized                                         True
  Ready                                               True
  ContainersReady                                     True
  PodScheduled                                        True
Volumes:                                              <none>
```

그림 3-5. Pod 2+2 Ready, AZ 분산

4. 검증

핵심 검증 9개를 모두 통과했다. 무중단 측정은 0.5~1초 간격 curl 루프로 했다.

#	항목	결과
V1	외부 접속	ALB → web → was → RDS 전 구간 200, 주인과 펫 등록, 조회
V2	무중단 롤링	이미지 교체 3회, 200 아닌 응답 **3회 모두 0**
V3	HPA 확장과 축소	2→4→2, 실패 **0 / 474,912**, p95 92ms
V4	커넥션 풀 상한	was 4개 각각 **8**, 합계 32 (계산 최대 53 ≪ `max_connections` 630)
V5	NetworkPolicy	차단 3경로, 허용 2경로 의도대로
V6①	RDS 장애 조치(직접)	앱 영향 **약 10초**(요청 152 중 실패 1), Pod 재시작 0
V6②	RDS 장애 조치(Proxy)	앱 영향 **약 12~13초**(요청 314 중 실패 2), Pod 재시작 0
V7	노드 drain	200 아닌 응답 **0**, PDB 유지
V8	야간 중지 후 복귀	**10분 16초**, ALB 주소 동일
V9	이미지 관문	옛 nginx 베이스 CRITICAL 9건 → `NOT pushed`

V1~V3는 처음 측정에서 문제가 드러났다. V1은 was가 기동하지 못했고, V2는 롤링 뒤 Pod가 한 AZ에 모였고, V3는 부하 중 502가 났다. 아래 세 건은 원인을 찾아 고친 뒤 다시 잰 기록이다.

4.1 V1 외부 접속과 기동 실패 (T3)

첫 이미지 1.0.0의 was는 startup probe에서 `Connection refused`로 계속 재시작했다. 원인은 Maven 필터링이었다. 원본 설정의 `${jdbc.url}`이 빌드 때 `localhost`로 박혀 실행 시 주입한 값이 먹지 않았다. 키 이름을 `pc.jdbc.*`로 바꾸고(3.3), 빌드 스크립트에 WAR 안에 자리표시가 남았는지 검사하는 단계를 넣어 1.0.1로 다시 빌드했다. 이후 주인과 펫 등록이 브라우저에서 정상 동작했다.

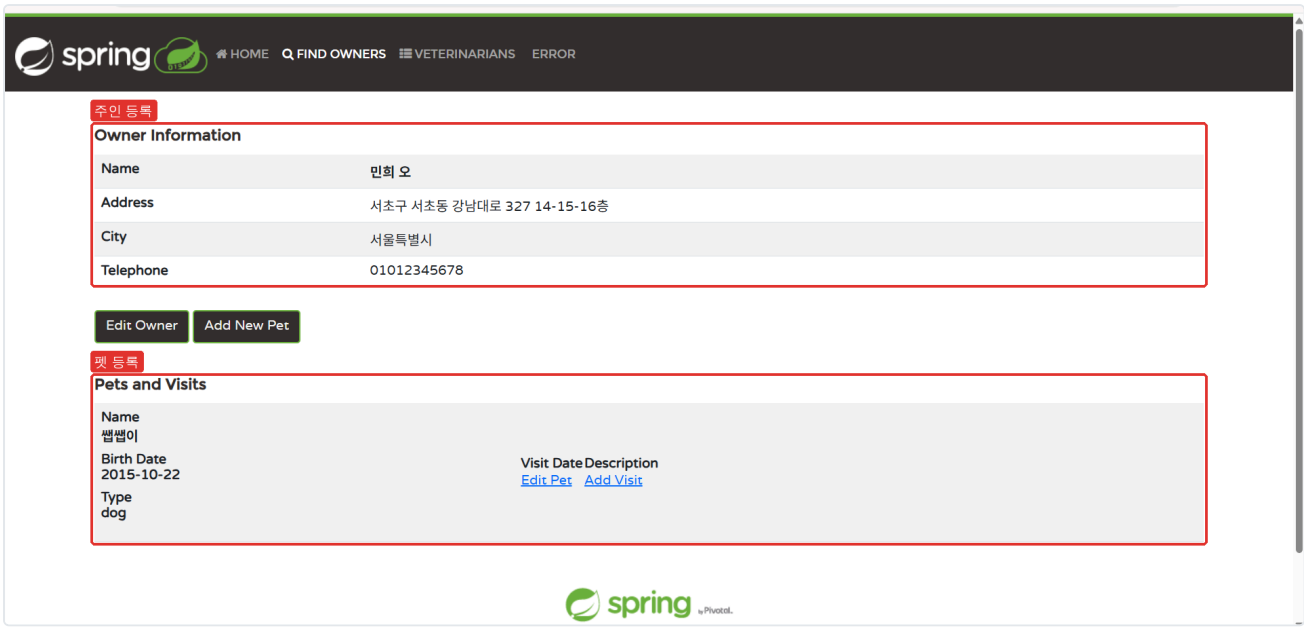


그림 4-1. 브라우저 PetClinic 화면 (주인과 펫 등록)

4.2 V2 무중단 롤링과 AZ 쏠림 (T5)

1차 측정에서 응답 실패는 0이었지만, 롤링이 끝나자 web 2개가 한 AZ에 모여 있었다. 이 상태로 그 AZ가 죽으면 서비스가 멈춘다. topologySpread가 사라질 옛 Pod까지 세었기 때문이다. `matchLabelKeys: [pod-template-hash]`를 넣어 새 ReplicaSet의 Pod만 세게 했고, 재측정에서 실패 0과 AZ 분산을 함께 확인했다.

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ export PC_OWNER=kdt11
bash scripts/90-verify.sh loop v2
[14:33:56] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[14:33:57] http://k8s-petclini-petclini-9d1e240fe8-433881641.ap-northeast-2.elb.amazonaws.com/owners?lastName= — Ctrl-C 로 종료

요청 276 · 200 아님 0   200 아닌 응답 0
200 아닌 코드별:
기록: /c/Users/5alsg/Desktop/k8s-project/logs/verify-20261026/v2-143357.tsv
```

그림 4-2. 롤링 배포 중 curl 루프: 요청 276, 200 아님 0

롤링 배포 3회와 노드 drain, drain 뒤 재배치(uncordon 후 rollout restart)까지 다섯 번의 측정을 합치면 요청 983건 중 200 아닌 응답은 0건이고, 응답 시간은 모두 1.5초 안쪽이었다(최대 1.35초).

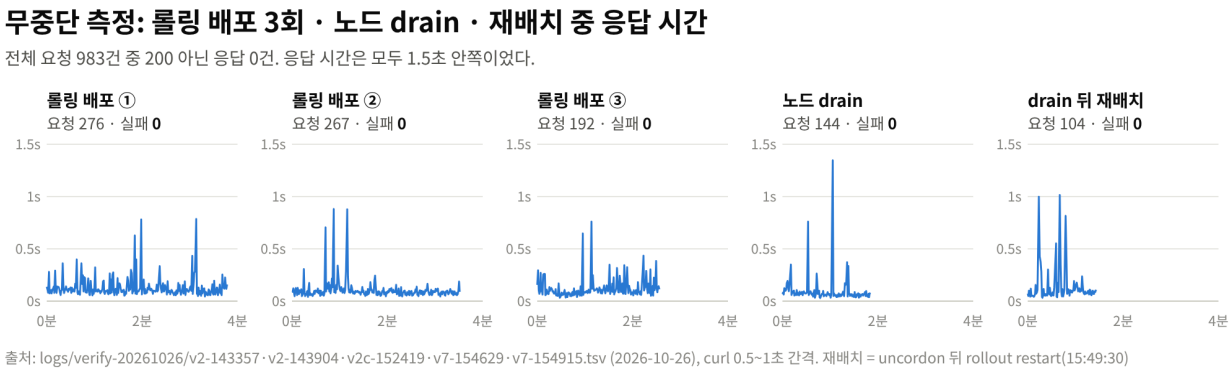


그림 4-3. 무중단 측정 5회의 응답 시간

4.3 V3 부하와 Target 5XX (T6)

1차 k6 실패율은 34.92%였다. 분해해 보니 대부분은 시험 스크립트가 없는 주소(`/vets.html`)를 요청해 생긴 404였고, 진짜 문제는 그 안의 Target 5XX 3.5%였다. nginx 오류 로그에는 `connect() failed (99: Address not available)` 가 찍혔다. nginx가 요청마다 was로 새 연결을 열고 닫아, 닫힌 연결이 TIME_WAIT 상태로 임시 포트를 붙잡으면서 포트가 바닥난 것이다. upstream keepalive(32개, 15초 < Tomcat 20초)를 넣은 1.0.3에서 Target 5XX는 0이 됐다. 시험 스크립트의 주소도 `/vets` 로 고친 2차 측정에서는 474,912건 실패 0, p95 92ms가 나왔다. was는 CPU 359%에서 즉시 2→4로 늘었고, 부하 종료 약 6분 뒤 2개로 돌아왔다.

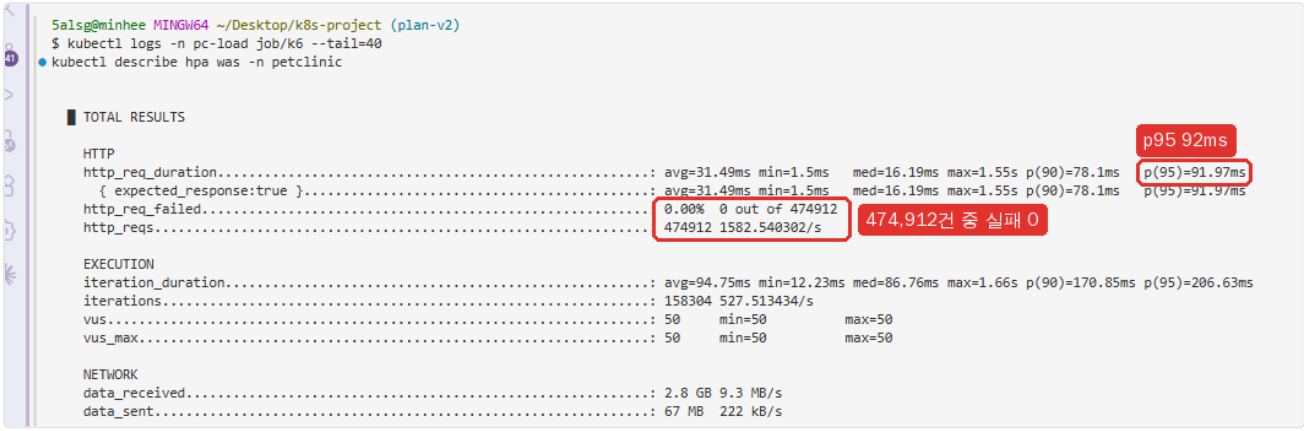


그림 4-4. k6 결과 (474,912건 실패 0, p95 92ms)

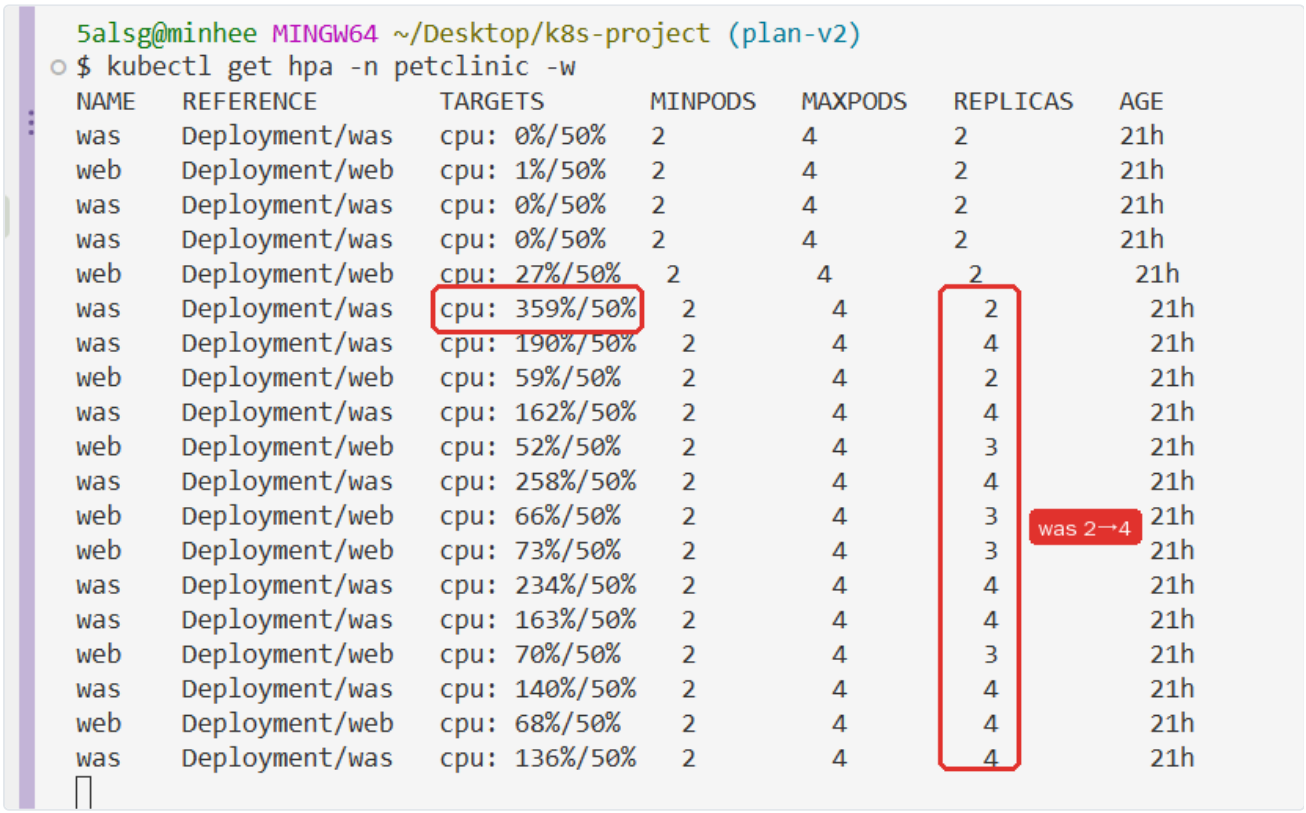


그림 4-5. 부하 중 HPA: was CPU 359% → replicas 2→4, web 2→3→4

4.4 그 밖의 검증 (V4, V5, V7, V9)

결과는 4장 표와 같고, 아래는 그 화면이다. 근거는 V4 연결 수 계산이 6.2, V5 정책과 V7 PDB가 3.4, V9 관문이 3.3에 있다. V6(RDS 장애 조치)은 5.1, V8(야간 중지 후 복귀)은 6.1에서 다룬다.

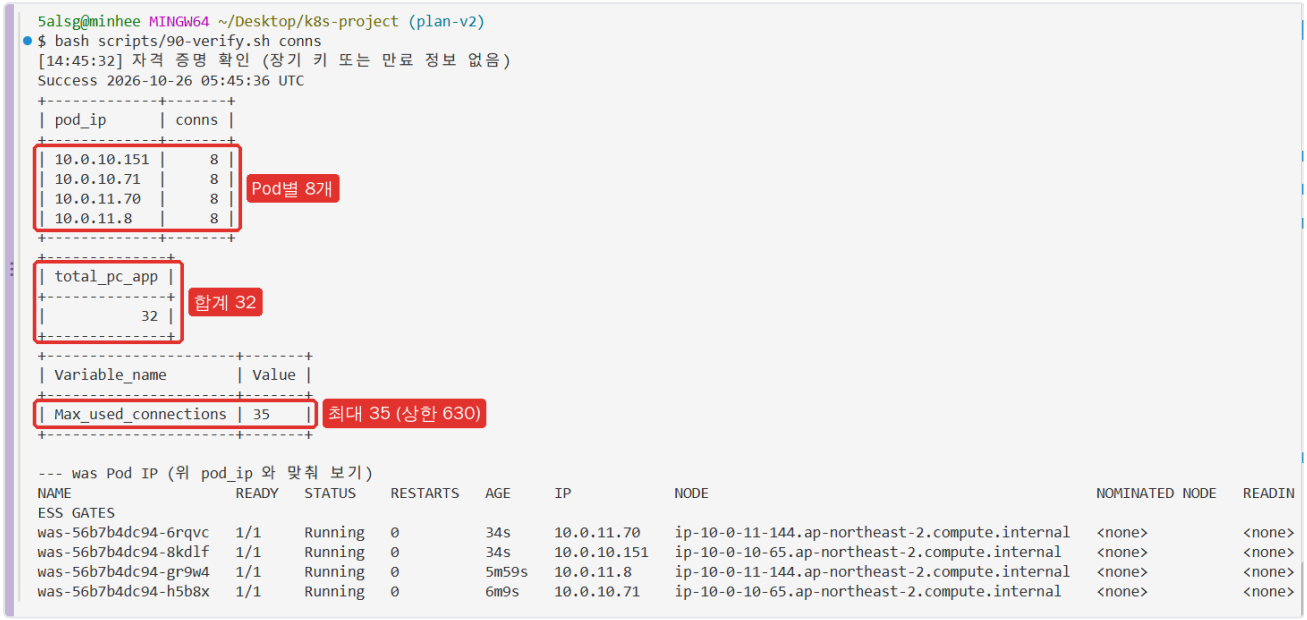


그림 4-6. 부하 중 was Pod 4개의 DB 연결 각 8개, 합계 32, Max_used_connections 35

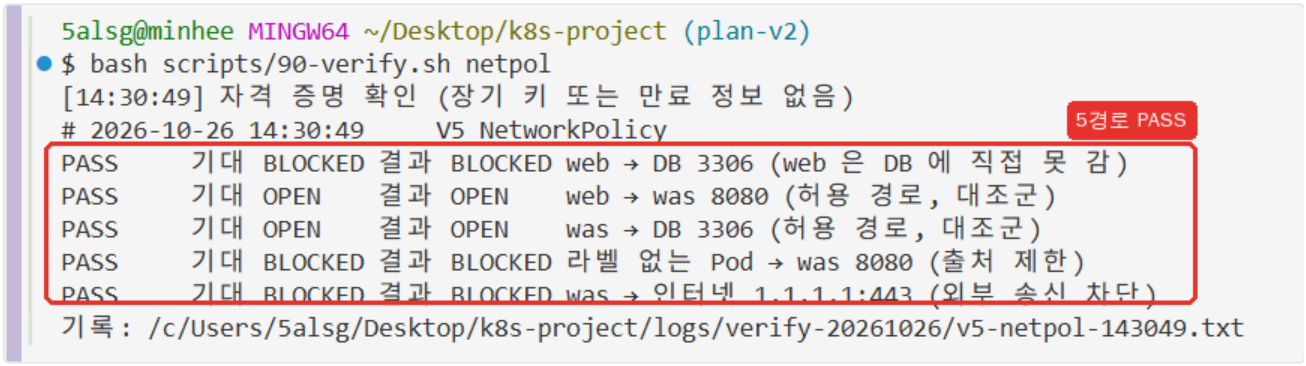


그림 4-7. NetworkPolicy 허용과 차단 5경로 PASS



그림 4-8. 2a 노드 drain (Pod 축출)

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/90-verify.sh loop v7
[15:46:28] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[15:46:29] http://k8s-petclini-petclini-9d1e240fe8-433881641.ap-northeast-2.elb.amazonaws.com/owners?lastName= — Ctrl-C 로 종료

요청 144 · 200 아님 0
200 아닌 코드별:
기록: /c/Users/5alsg/Desktop/k8s-project/logs/verify-20261026/v7-154629.tsv
```

200 아닌 응답 0

그림 4-9. drain 중 curl 루프: 요청 144, 200 아님 0

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ WEB_BASE=nginxinc/nginx-unprivileged:1.20.0-alpine IMAGE_TAG=v9-gate bash scripts/60-build-push.sh 2>&1 | tee logs/v9-gate.log
[15:40:56] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[15:40:59] 앱 커밋 cf392920cdbe9cd07689f1f4b603271940bac957 → 태그 v9-gate
[15:41:00] SSM 명령 9c733824-de73-47ee-8c92-6eff6a6128d0 전송 — 빌드 대기 (첫 빌드는 Maven 의존성 때문에 5~10분)
Failed
== source (cf392920cdbe9cd07689f1f4b603271940bac957)
petclinic commit: cf392920cdbe9cd07689f1f4b603271940bac957 2026-10-05 Bump org.springframework.data:spring-data-bom from 2025.1.5 to 2026.0.1 (#296)
pom patched: com.mysql:mysql-connector-j:26.7.0
datasource patched: pc.jdbc.* placeholders, pool maxActive=8

== WAR (Maven container, -PMySQL, tests skipped)
-rw-r--r--. 1 root root 51M Oct 26 06:41 app/target/petclinic.war
built WAR keeps ${pc.jdbc.*} placeholders

== build context

== docker build
sha256:f616809d6be196f5aab8414b531db0450cd3f386093dc973d466ba24e8f6fb64
sha256:77e2aae64f085eba08ffb23d95307506839f77e55751bb4402a9815fab838162
[redacted]r.ap-northeast-2.amazonaws.com/pc-was:v9-gate  432MB
[redacted]r.ap-northeast-2.amazonaws.com/pc-web:v9-gate  23.6MB

== Trivy gate (CRITICAL, fixable only)
-- pc-web
[redacted]r.ap-northeast-2.amazonaws.com/pc-web:v9-gate (alpine 3.13.5)
==========
Total: 9 (CRITICAL: 9)
```

CRITICAL 9건

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apk-tools	CVE-2021-36159	CRITICAL	fixed	2.12.5-r0	2.12.6-r0	libfetch: an out of boundary read while libfetch uses strtol to parse... https://avd.aquasec.com/nvd/cve-2021-36159
curl	CVE-2021-22945			7.76.1-r0	7.79.0-r0	curl: use-after-free and double-free in MQTT sending https://avd.aquasec.com/nvd/cve-2021-22945
	CVE-2022-32207				7.79.1-r2	curl: Unpreserved file permissions https://avd.aquasec.com/nvd/cve-2022-32207
freetype	CVE-2022-27404			2.10.4-r1	2.10.4-r2	FreeType: Buffer overflow in sfnt_init_face https://avd.aquasec.com/nvd/cve-2022-27404
libcrypto1.1	CVE-2021-3711			1.1.1k-r0	1.1.1l-r0	openssl: SM2 Decryption Buffer Overflow https://avd.aquasec.com/nvd/cve-2021-3711
libcurl	CVE-2021-22945			7.76.1-r0	7.79.0-r0	curl: use-after-free and double-free in MQTT sending https://avd.aquasec.com/nvd/cve-2021-22945
	CVE-2022-32207				7.79.1-r2	curl: Unpreserved file permissions https://avd.aquasec.com/nvd/cve-2022-32207
libssl1.1	CVE-2021-3711			1.1.1k-r0	1.1.1l-r0	openssl: SM2 Decryption Buffer Overflow https://avd.aquasec.com/nvd/cve-2021-3711
zlib	CVE-2022-37434			1.2.11-r3	1.2.12-r2	zlib: heap-based buffer over-read and overflow in inflate() in inflate.c via a... https://avd.aquasec.com/nvd/cve-2022-37434

```
-- pc-web exit=1
-- pc-was
   pc-was exit=1
STOP: Trivy gate failed - fixable CRITICAL vulnerabilities. NOT pushed
```

push 중단

```
IMAGE_TAG=v9-gate
[15:41:35] 중단: 빌드 실패(Failed) — 위 출력 확인
```

그림 4-10. 옛 nginx 베이스: Trivy CRITICAL 9건 → NOT pushed (계정 ID 가림)

5. 심화 실험

4장 검증에 더해 세 가지를 실험했다. 5.1은 V6(RDS 장애 조치)을 RDS Proxy와 비교하도록 넓힌 것이고, 5.2(web과 was를 묶은 카나리)와 5.3(시점 복구)은 새로 한 실험이다.

5.1 직접 연결 vs RDS Proxy (V6)

이 구성에서는 RDS Proxy가 장애 조치 시간을 줄였다는 근거를 얻지 못했다.

- **수치**: 앱 영향은 직접 연결 약 10초(요청 152 중 실패 1), Proxy 약 12~13초(요청 314 중 실패 2)였다. 요청 로그로 다시 재면 첫 실패 요청 시작부터 마지막 지연 응답까지 직접 9.4초, Proxy 12.3초다(그림 5-1).
- **한계**: 두 값은 각 1회 측정이고 curl 간격이 0.5~1초라, 이 해상도에서는 차이가 없다고만 말할 수 있다.
- **직접 연결이 이미 빨랐던 이유**: JVM DNS 캐시를 30초로 줄였고, 풀이 `SELECT 1`로 끊긴 연결을 버린다.
- **Proxy로 얻은 것**: 연결 집약이다. RDS에서 본 앱 쪽 연결 수는 직접 연결일 때 was Pod당 최대 8(V4 부하 중 4 Pod 합계 32), Proxy 경유일 때 Proxy에서 온 연결 4였다. 후자는 부하 없이 was 2개일 때 값이라 같은 조건의 비교는 아니다.
- **결론**: Pod가 많아져 커넥션이 상한이 될 때 도입할 수단이다.

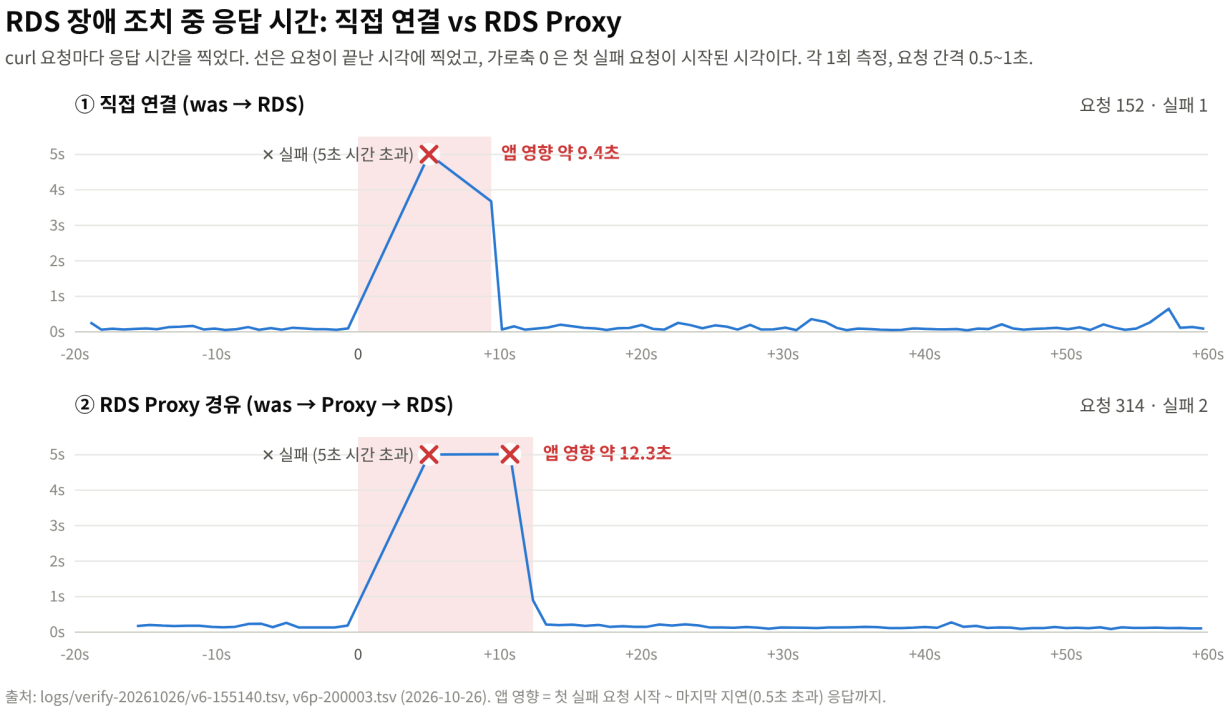


그림 5-1. RDS 장애 조치 중 응답 시간 (직접 연결 vs RDS Proxy)

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/90-verify.sh loop v6
[15:51:39] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[15:51:40] http://k8s-petclini-petclini-9d1e240fe8-433881641.ap-northeast-2.elb.amazonaws.com/owners?lastName= — Ctrl-C 로 종료
15:52:05.048    000     5.004292   5초 타임아웃 1건

요청 152 · 200 아님 1 (처음 15:52:05.048 ·마지막 15:52:05.048)   152건 중 실패 1
200 아닌 코드별:
      1 000
기록: /c/Users/5alsg/Desktop/k8s-project/logs/verify-20261026/v6-155140.tsv
```

그림 5-2. 직접 연결, 장애 조치 중 요청 기록

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/90-verify.sh loop v6p
[20:00:01] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
[20:00:03] http://k8s-petclini-petclini-abf1810263-705015855.ap-northeast-2.elb.amazonaws.com/owners?lastName= — Ctrl-C 로 종료
20:00:24.110    000     5.003824
20:00:29.827    000     5.013054   타임아웃 2건

요청 314 · 200 아님 2 (처음 20:00:24.110 ·마지막 20:00:29.827)   314건 중 실패 2
200 아닌 코드별:
      2 000
기록: /c/Users/5alsg/Desktop/k8s-project/logs/verify-20261026/v6p-200003.tsv
```

그림 5-3. RDS Proxy 경유, 장애 조치 중 요청 기록

5.2 수직 묶음 카나리

새 버전 1.1.0을 기존 1.0.4 옆에 띄웠다(1.0.4는 카나리 묶음을 위해 nginx upstream을 환경 변수로 바꾼 재빌드이고, 앱 동작은 1.0.3과 같다). web-canary로 간 요청은 반드시 was-canary로 간다. 헤더 선공개에서는 `x-canary` 요청 20/20이 새 버전으로 갔다. 가중치 10%에서는 요청 200건 중 19건(9.5%)이 새 버전이었고 5xx는 0이었다. `down` 뒤에는 stable 100%로 돌아왔다.

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/75-canary.sh measure
[19:23:54] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
# 2026-10-26 19:23:55 카나리 측정 — 헤더 없음 200번, x-canary 20번
구분  응답코드  버전  개수
header  200  1.1.0  20   헤더: 20/20 새 버전
plain  200  1.0.4  181
plain  200  1.1.0  19   가중치 10%: 19/200
기록: /c/Users/5alsg/Desktop/k8s-project/logs/verify-20261026/canary-192355.tsv
```

그림 5-4. 카나리 버전별 응답 수 (1.1.0 / 1.0.4)

5.3 PITR 복구 리허설

시점 T(owners 10)를 기록한 뒤 주인 1명을 추가하고, T로 새 인스턴스를 복원했다. 복원에는 11분이 걸렸고, 복원본은 owners 10으로 T 뒤의 변경이 없었다. 복원은 항상 새 인스턴스라 실제 복구라면 `db` 전환이 더 필요하다. RTO는 복원 11분에 전환 시간 몇 분을 더해 잡는다. RPO는 트랜잭션 로그 업로드 주기로 정해진다. T(10:40:23 UTC)를 찍은 뒤 복원 가능 최신 시점(LatestRestorableTime)이 T를 넘은 것은 10:44:43이었다. 즉 복원 가능 시점이 실제 시각보다 약 4~5분 늦고, RPO는 최대 약 5분이다.



그림 5-5. PITR 결과. 운영 owners 11, 복원본 10

6. 운영

6.1 운영 절차

작업	명령	결과
새 버전 배포(롤링)	`IMAGE_TAG=<태그> 60-build-push.sh` → `70-deploy.sh`	V2: 3회, 실패 0
확장과 축소	없음 (HPA 자동)	V3: 2→4→2
카나리	`75-canary.sh up <태그>` → `weight 10` → `down`	5.2
노드 정비	`kubectl drain <노드> --ignore-daemonsets --delete-emptydir-data` → `uncordon` → `rollout restart`	V7: 실패 0
DB 경로 전환	`76-rds-proxy.sh use` / `direct` (`db` 한 줄 교체 + was 재시작)	V6②
DB 시점 복구	`77-pitr.sh mark` → `restore` → `check` → `delete`	5.3
야간 중지와 기동	`stop.sh` (앱 0 → 노드 0 → RDS, 관리 EC2) / `start.sh` (역순)	V8: 10분 16초

- **카나리 되돌리기 순서**: HTTPRoute를 먼저 stable 100%로 돌리고, 30초 뒤 카나리를 지운다. 반대로 하면 진행 중인 요청이 사라진 대상에 닿는다.
- **롤백**: 이전 태그로 다시 배포한다. 태그를 바꿀 수 없으므로 이전 태그는 검증했던 그 이미지다. `rollout undo`는 매니페스트와 클러스터가 어긋나서 쓰지 않았다. 롤백 자체는 실측하지 않았다.



그림 6-1. 기동 스크립트 실행 10분 16초 후 ALB 200

6.2 용량 산정

현재 구성은 초당 약 1,580 요청을 실패 0으로 처리했다. 확장 상한은 DB 커넥션이 아니라 노드 2대의 CPU다.

- **노드**: HPA 최대(was 4 × 500m + web 4 × 100m = 2.4 vCPU, 카나리 중이면 +0.6 = 3.0)를 노드 1대에 올리는 N+1을 목표로 했고, 그래서 노드 자동 확장은 넣지 않았다. 다만 노드가 실제로 쓸 수 있는 양(allocatable)은 4 vCPU보다 작고(EKS AMI 예약 공식으로 약 3.9), 같은 노드에 CoreDNS, LBC, ESO, metrics-server, aws-node, kube-proxy의 request도 올라간다. 철거 전에 `kubectl describe nodes`의 Allocated resources를 남기지 않았으므로 **이 계산은 시스템 Pod를 뺀 추정**이다. 실측으로 확인된 것은 최소 개수(2+2)에서 노드 1대를 drain해도 실패 0(V7)이라는 점까지다.
- **DB 연결**: (HPA 4 + surge 1 + canary 1) × 풀 8 + 관리 5 = 53 ≪ `max_connections` 630. 부하 중 실측은 Pod별 8개, 합계 32에서 더 늘지 않았다.
- **HPA**: 축소에 약 6분이 걸린 것은 기본 축소 안정화 시간 300초 때문이다(4.3).

6.3 보안 점검

인터넷에 열린 것은 ALB 80 하나다.

영역	항목	결과
네트워크	EKS API 퍼블릭 비노출, DB 서브넷 인터넷 경로 없음, 관리 EC2 인바운드 0	적합
네트워크	RDS 3306은 노드, 관리 EC2, Proxy SG에서만, Pod 간 기본 차단(V5)	적합
IAM	Pod 정적 키 없음(Pod Identity 2개), Pod가 노드 역할 사용 불가	적합
비밀	코드, 매니페스트, 로그에 비밀번호 없음, master 비밀은 클러스터 밖	적합 (was 컨테이너 안 설정 파일에는 평문, 3.3)
DB	앱 계정 DML만, TLS 강제, 저장 암호화, 백업 7일	적합 (인증서 검증 미적용)
이미지	push 전 취약점 차단(V9), 태그 불변, 연속 스캔	적합 (직접 push 우회 가능)
Pod	PSS baseline 강제, restricted 경고 0, 비루트, cap drop ALL	적합
전송	사용자 ↔ ALB HTTPS	**미적용** (도메인 없음)

6.4 비용과 철거

가장 큰 절감은 자원이 존재하는 시간을 줄이는 것이었다. 밤에는 노드, RDS, 관리 EC2를 끄고, 철거는 측정을 마친 직후(10/26)에 했다. 야간 중지는 테스트 기간의 비용 조치이고, 운영 기준은 24시간 가동이다. 설계(NAT, 노드 수, Multi-AZ)는 24시간 가동을 전제로 정했다.

실제 청구는 10/22~10/27 약 $32다(Cost Explorer, 그림 6-2). 하루 최대는 약 $12(10/25, UTC 기준)이고, 큰 항목은 NAT 시간 요금, EKS 컨트롤 플레인, RDS Multi-AZ, 노드다. 밤에도 남긴 EKS 컨트롤 플레인, NAT, EIP, ALB(시간당 약 $0.27)가 매일 깔리는 바닥 비용이었다. 다시 만들면 엔드포인트와 주소가 바뀌어서 남겼다.

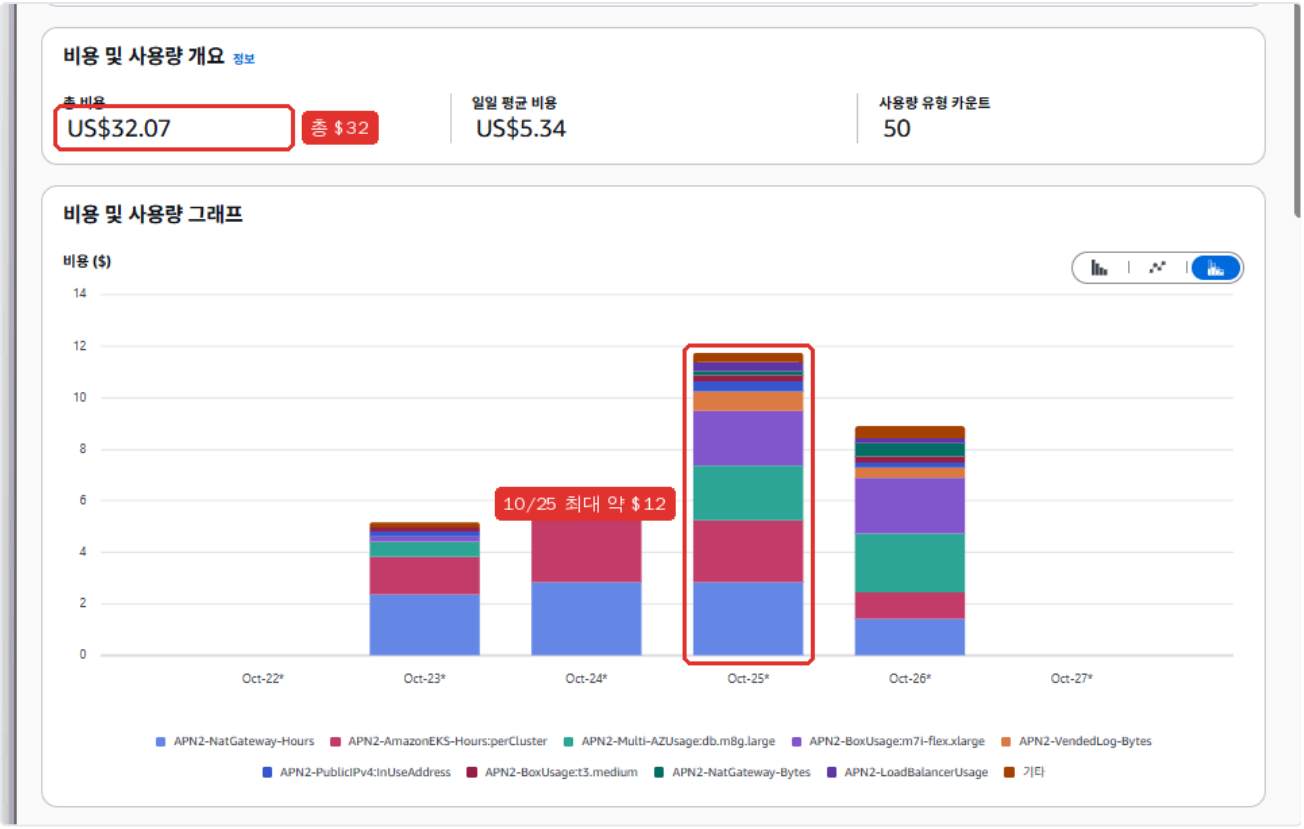


그림 6-2. 실제 비용 (Cost Explorer, 10/22~10/27)

설계 기준대로 24시간 운영하면 드는 월 비용은 아래와 같다(서울 온디맨드, 730시간, Price List 2026-10-22 게시본 기준 추정).

항목	구성	월 비용
EKS 노드	m7i-flex.xlarge × 2	약 $344
RDS	db.m8g.large Multi-AZ	약 $342
NAT	2개 (AZ마다)	약 $86
EKS 컨트롤 플레인	1	약 $73
관리 EC2	t3.medium	약 $38
기타	ALB, EIP, EBS와 RDS 스토리지, Secrets Manager, ECR	약 $40
합계		**약 $920**

철거 전에는 확인 관문을 통과해야 한다. 철거하면 다시 찍을 수 없기 때문이다.

#	항목	통과 기준
E1	V1~V9 결과	측정값과 스크린샷 확보
E2	`80-evidence.sh` 구성 덤프	덤프 생성, 마스킹 검사 0건
E3	외부 접속 화면 + `curl -I` 200	캡처
E4	콘솔: VPC 리소스 맵, EKS 네트워킹, RDS 구성, ALB 규칙, CloudWatch	캡처, 계정 ID 가림
E5	아키텍처 그림이 덤프와 일치	다르면 그림을 고친다

`destroy.sh`는 터미널에서 `pc-eks`를 입력해야만 실행된다. 지우는 순서는 Gateway와 ALB → Helm → 노드 그룹과 클러스터(터널은 끝까지 유지, T9) → RDS Proxy와 RDS(최종 스냅샷) → IAM → 관리 EC2와 ECR → 비밀 → NAT와 VPC 순이다. 남긴 것은 최종 스냅샷 2개뿐이고, 2026-11-04에 삭제한다.

```
5alsg@minhee MINGW64 ~/Desktop/k8s-project (plan-v2)
$ bash scripts/destroy.sh
[16:39:06] 자격 증명 확인 (장기 키 또는 만료 정보 없음)
pc-* 자원을 모두 삭제합니다 (RDS 는 최종 스냅샷을 남김).
증빙 체크리스트 E1~E5 를 모두 통과했는지 먼저 확인하세요.
계속하려면 클러스터 이름(pc-eks)을 입력: pc-eks
[16:39:13] 관리 EC2 없음 · ALB 0 — 쿠버네티스 정리(1·2단계)는 끝난 것으로 보고 건너뜀
[16:39:13] 클러스터 삭제 (15~20분)
[16:39:14] 스택 삭제: eksctl-pc-eks-podidentityrole-kube-system-aws-load-balancer-controller
[16:39:48] ✓ eksctl-pc-eks-podidentityrole-kube-system-aws-load-balancer-controller
[16:39:48] 스택 삭제: eksctl-pc-eks-podidentityrole-external-secrets-external-secrets
[16:40:21] ✓ eksctl-pc-eks-podidentityrole-external-secrets-external-secrets
[16:40:22] 스택 삭제: eksctl-pc-eks-accessentry-7TGVLO6F6KFNN7AGXZC2WSGT3O2S4LJT
[16:40:56] ✓ eksctl-pc-eks-accessentry-7TGVLO6F6KFNN7AGXZC2WSGT3O2S4LJT
[16:40:57] 스택 삭제: eksctl-pc-eks-nodegroup-pc-ng-app
[16:50:04] ✓ eksctl-pc-eks-nodegroup-pc-ng-app
[16:50:04] 스택 삭제: eksctl-pc-eks-cluster
[16:52:38] ✓ eksctl-pc-eks-cluster
[16:52:40] ✓ EKS 클러스터
[16:52:43] RDS 상태 none — 삭제 요청 생략
[16:52:48] ✓ ECR 레지스트리 스캔 BASIC
[16:52:50] ✓ IAM 역할 pc-role-lbc
[16:52:52] ✓ IAM 역할 pc-role-eso
[16:52:54] ✓ IAM 역할 pc-role-mgmt
[16:52:55] ✓ IAM 역할 rds-monitoring-role
[16:53:01] ✓ 로그 그룹 /aws/eks/pc-eks/cluster
[16:53:03] ✓ 로그 그룹 /aws/rds/instance/pc-rds/error
[16:53:13] ✓ SG sg-08fa01c4f6c09552f
[16:53:16] ✓ 서브넷 subnet-0dd10cbda5a46f1d4
[16:53:17] ✓ 서브넷 subnet-084a520572f505f43
[16:53:18] ✓ 서브넷 subnet-0d5d5a12b0adc622a
[16:53:20] ✓ 서브넷 subnet-09d4b4707843074dc
[16:53:22] ✓ 라우트 테이블 rtb-0f45e04f32853527c
[16:53:23] ✓ 라우트 테이블 rtb-03f039928e809a69f
[16:53:24] ✓ 라우트 테이블 rtb-03e41ee6b86cd25ae
[16:53:25] ✓ 라우트 테이블 rtb-0d8c3041c9c874dc7
[16:53:27] ✓ VPC
== 남은 Project=petclinic 태그 자원 (삭제 직후 몇 분은 보일 수 있음. 스냅샷 1개는 정상)
arn:aws:ec2:ap-northeast-2:************:volume/vol-050ee147c4c6e7e47
arn:aws:ec2:ap-northeast-2:************:network-interface/eni-00be933e91b53c5c0
arn:aws:ec2:ap-northeast-2:************:volume/vol-06c71453284d89881
arn:aws:ec2:ap-northeast-2:************:instance/i-00afefe299f96db2a
arn:aws:secretsmanager:ap-northeast-2:************:secret:pc/petclinic/app-db-CPMGAz
arn:aws:ec2:ap-northeast-2:************:vpc-endpoint/vpce-0c1901debd9a6f590
arn:aws:rds:ap-northeast-2:************:snapshot:pc-rds-final-20261026
arn:aws:ec2:ap-northeast-2:************:instance/i-0e4ddd56a00393e39
arn:aws:ec2:ap-northeast-2:************:network-interface/eni-03e83d2f815bbfbbf
arn:aws:ec2:ap-northeast-2:************:natgateway/nat-0a7f1f7ec16a8d7b7
arn:aws:ec2:ap-northeast-2:************:volume/vol-0407d894dcb166cea
arn:aws:ec2:ap-northeast-2:************:natgateway/nat-0955d85b150ed9f0c
arn:aws:ec2:ap-northeast-2:************:instance/i-0372345b5308f4894
arn:aws:secretsmanager:ap-northeast-2:************:secret:pc/petclinic/migration-db-IUsuEx
== 최종 스냅샷
pc-rds-final-20261026   available       2026-10-26T07:30:23.721000+00:00
== 로컬 정리(수동): hosts 의 EKS 핸드로한드 줄, ~/.kube/config 의 pc-eks 컨텍스트
== 리소스 대장: 스냅샷 pc-rds-final-20261026 삭제일 2026-11-04
[16:53:29] 정리 완료
```

VPC까지 삭제

최종 스냅샷

정리 완료

그림 6-3. 1차 철거 스크립트 출력 (계정 ID는 ***로 가림). 최종 철거도 같은 순서

7. 회고와 개선 과제

가장 큰 수확은 첫 배포에서는 보이지 않던 문제를 측정으로 찾아 고친 것이다(4장).

- **측정은 분해해서 본다.** k6 실패율 34.92%는 대부분 시험 스크립트의 404였고, 진짜 문제는 그 안의 Target 5XX 3.5%였다(4.3).
- **개선이라고 생각한 것도 재어 본다.** RDS Proxy로 장애 조치가 빨라질 것이라 기대했지만, 빨라졌다는 근거는 없었다(5.1).
- **기본값과 짐작에 기대지 않는다.** metrics-server는 eksctl 기본값에 기대다 재구축에서 빠졌고(T11), 카나리에서는 LBC가 만든 TGB 이름을 짐작했다가 대기에서 멈췄다(T12). 컨트롤러가 만든 자원은 이름이 아니라 스펙 필드로 찾는다.
- **철거도 설계 대상이다.** 접속 경로는 마지막까지 살려 둔다(T9).
- **다시 돌려도 안전한 스크립트가 일정을 구했다.** 철거 후 같은 날 재구축해 심화 실험까지 마쳤다.

영역	지금의 한계	개선
이미지 공급망	스크립트 관문이라 직접 push로 우회 가능	Kyverno로 다이제스트 고정과 서명 검증
인프라 코드	Bash와 AWS CLI 스크립트(Terraform은 다음 학습 단계), 상태 관리 없이 태그 조회로 재실행	Terraform 모듈로 전환해 상태 관리와 드리프트 감지
배포	관리 EC2에서 스크립트 실행	GitHub Actions(OIDC) → Argo CD, 카나리는 Argo Rollouts
전송 보안	HTTP 80(도메인 없음), DB 인증서 미검증	ACM HTTPS, `VERIFY_IDENTITY`
관측	`kubectl logs`뿐, ALB 접근 로그 없음	Container Insights와 Fluent Bit, ALB 로그, 5xx 경보
경계와 Pod 보안	WAF 없음, PSS restricted는 경고만	AWS WAF, restricted 강제
비밀과 접근	비밀번호 수동 교체, 생성자 관리자 권한 유지	자동 교체, 운영자 역할 액세스 엔트리만
확장과 복구	노드 2대 고정, PITR 후 전환 수동	Karpenter, 복원과 전환 스크립트와 정기 리허설
확장 지표	HPA가 CPU만 본다. 이번 부하는 CPU로 바로 반응했지만, was가 DB를 기다리는 부하에서는 CPU가 낮아 늦을 수 있다(미측정)	KEDA로 ALB 요청 수(CloudWatch) 기준 확장 추가
네임스페이스 자원 한도	매니페스트마다 직접 적은 requests에만 의존해, 빠뜨려도 막지 않는다	LimitRange로 기본 request, ResourceQuota로 네임스페이스 합계 상한

부록

A. 리소스 명명

모든 AWS 자원은 `pc-<종류>-<역할>-<AZ>` 규칙을 따르고, 태그 `Project=petclinic`, `Owner`, `Env=prod`를 붙였다. 접두사를 짧게 잡은 것은 ALB와 대상 그룹 이름이 32자로 제한되기 때문이다.

영역	이름
네트워크	`pc-vpc`, `pc-subnet-{pub,app,db,mgmt}-{a,c}`, `pc-nat-{a,c}`, `pc-sg-{mgmt,node,rds,proxy}`
컴퓨트	`pc-mgmt`, `pc-eks`, `pc-ng-app`, 역할 `pc-role-{mgmt,lbc,eso,rds-proxy}`
데이터	`pc-rds`, `pc-proxy`, `pc-rds-pitr`, 비밀 `pc/petclinic/{app,migration}-db`
이미지	ECR `pc-web`, `pc-was`, 태그 `<앱 버전>-<커밋>` (예: `1.0.3-cf39292`), 태그 변경 불가(IMMUTABLE)
쿠버네티스	네임스페이스 `petclinic`, `web`, `was` (+ `-canary`), Service `db` (ExternalName), Gateway `petclinic`

B. 버전

대상	버전
EKS, 노드	1.36, 관리형 노드 그룹 m7i-flex.xlarge × 2 (AZ마다 1대)
RDS	MySQL 8.4.11, db.m8g.large, Multi-AZ
애드온	LBC 3.5.0, Gateway API CRD v1.6.0, ESO 2.12.0, metrics-server v0.9.0-eksbuild.13(재구축 후 .14, EKS 애드온), vpc-cni(NetworkPolicy 사용)
이미지	`tomcat:11.0.27-jre21-temurin-noble`, `nginxinc/nginx-unprivileged:1.30.6-alpine`
드라이버	`com.mysql:mysql-connector-j` 26.7.0

버전은 릴리스 후 2주가 지난 것 중 최신판을 골랐다.

C. 스크립트 목록

파일	하는 일
`env.sh`, `lib.sh`, `01-preflight.sh`	공통 변수와 함수, 사전 확인
`10-network`, `20-mgmt-ec2`, `21-tunnel`	네트워크(VPC, NAT, SG), 관리 EC2, SSM 터널
`30-cluster`, `31-rds`, `40-addons`	EKS, RDS, 애드온(LBC, Gateway CRD)
`32-db-bootstrap`, `34-ecr`, `35-secrets-eso`	DB 계정, ECR, 비밀과 ESO
`60-build-push`, `70-deploy`	빌드→Trivy→push, 배포
`75-canary`, `76-rds-proxy`, `77-pitr`	카나리, RDS Proxy, PITR
`80-evidence`, `90-verify`	구성 덤프, 검증
`stop`, `start`, `destroy`	야간 중지와 기동, 철거

D. web nginx 설정 (발췌)

```
upstream was_backend {
    server ${WAS_UPSTREAM}:8080;    # stable = was, 카나리 web = was-canary
    keepalive 32;
    keepalive_timeout 15s;          # < Tomcat keepAliveTimeout 20s
    keepalive_requests 100;         # Tomcat maxKeepAliveRequests 기본 100
}
server {
    listen 8080;
    add_header X-App-Version "${APP_VERSION}" always;
    location = /healthz { access_log off; return 200 "ok\n"; }
    location /resources/ { root /usr/share/nginx/html; expires 1h; }   # 정적 파일은 web이 직접
    location / {
        proxy_pass http://was_backend;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
    }
}
```

E. 트러블슈팅

기록한 23건 중 보고서와 관련된 13건이다. 서비스 품질에 직접 닿은 T3, T5, T6은 4장에서 자세히 다뤘다.

#	증상	원인 → 조치
T1	`-PMySQL` 빌드가 드라이버를 못 찾음	원본 pom의 옛 좌표 → `com.mysql:mysql-connector-j`로 교체
T2	DB 주소를 바꿔도 `localhost`로 접속	Maven 필터링이 `${jdbc.url}`을 빌드 때 값으로 박음 → 자리표시 이름 변경
T3	was startup probe 실패 (`Connection refused`)	T2와 같은 필터링 잔존 → `pc.jdbc.*`로 바꾸고 빌드 후 WAR 검사, 1.0.1 재빌드 (4.1)
T4	Git Bash에서 비교와 인코딩 오류	CRLF와 cp949 → `lib.sh` 래퍼로 정규화
T5	롤링 후 web 2개가 한 AZ	topologySpread가 옛 Pod까지 셈 → `matchLabelKeys: [pod-template-hash]` (4.2)
T6	부하 중 Target 5XX 3.5%	nginx가 요청마다 was 연결을 새로 열어 임시 포트 고갈 → upstream keepalive, 1.0.3에서 0 (4.3)
T7	구성 덤프 마스킹 검사 실패	오탐과 ARN 파싱 오류 → 값이 붙은 경우만 검사, 파일 이름도 검사
T8	`stop.sh`가 중간에 멈춤	인터넷 끊김으로 빈 값을 "0 아님"으로 처리 → 조회 실패와 조건 미충족 구분
T9	1차 destroy에서 클러스터 삭제 중단	프라이빗 엔드포인트인데 터널을 먼저 닫음 → 접속 경로를 철거의 마지막으로
T10	재구축 중 PC 종료 → Pod Identity 0개	클러스터가 있으면 생성을 건너뜀 → 연결 생성 후 스크립트 재실행
T11	재구축 후 metrics-server 없음	eksctl 기본값에 기댐 → `cluster.yaml.tpl`에 명시
T12	카나리 `up`이 대기에서 멈춤	LBC가 TGB 이름을 잘라 만듦 → `serviceRef.name`으로 찾기
T13	`30-cluster.sh`가 SG 없음으로 중단	탭 구분 출력 비교 실패 → `xargs`로 정규화